WEEK 2 CHECKPOINT: MODELING LEAD PROGRESS

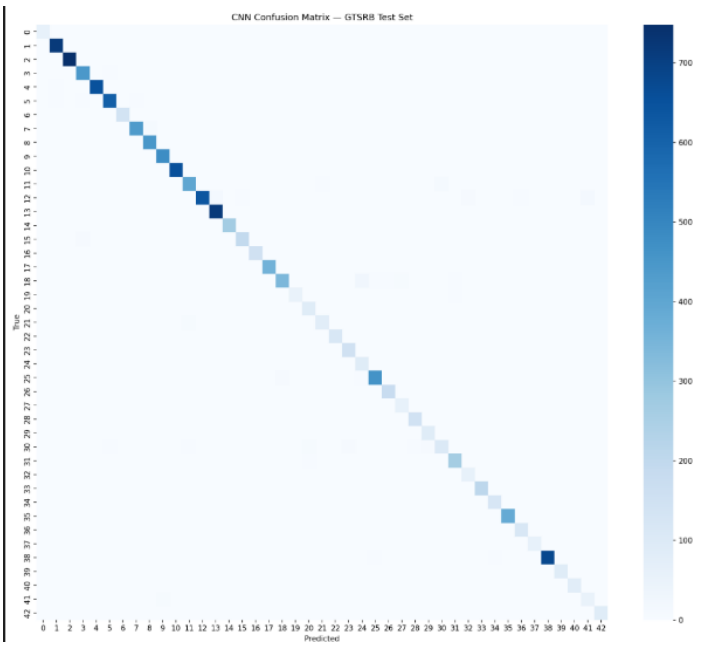
DATA PIPELINE AND BASELINES

A reproducible data pipeline was implemented (`src/data_pipeline.py`) to automatically download the German Traffic Sign Recognition Benchmark (GTSRB) dataset. This pipeline resizes images to a standard 32x32 resolution and applies data augmentations—including random rotations and color jittering—to prevent overfitting during convolutional neural network (CNN) training. The dataset was successfully partitioned into stratified training, validation, and testing sub-sections. To establish a comparative baseline, a simple non-deep-learning model was trained (`src/train.py`). A Support Vector Machine (SVM) utilizing Histogram of Oriented Gradients (HOG) features was implemented. This baseline successfully yielded an initial validation accuracy of 91.44% and a macro-F1 score of 90.06%, establishing a strong performance floor for the subsequent CNN experiment.

CNN EXPERIMENT RUNNING

In fulfillment of the requirement to build at least one model from scratch, a custom 3-block Convolutional Neural Network (CNN) architecture was designed and implemented (`src/models/cnn.py`). The CNN was trained on the GTSRB dataset over 15 epochs using an Adam optimizer and a learning rate scheduler.

The initial results of this CNN experiment demonstrated successful convergence. Evaluation on the test dataset resulted in a classification accuracy of 81.97% and a macro-F1 score of 74.23%. Furthermore, training logs, learning curve plots, and confusion matrix heatmaps were generated and saved to the repository (`experiments/results/`) to facilitate ongoing error and slice analysis in the coming weeks.



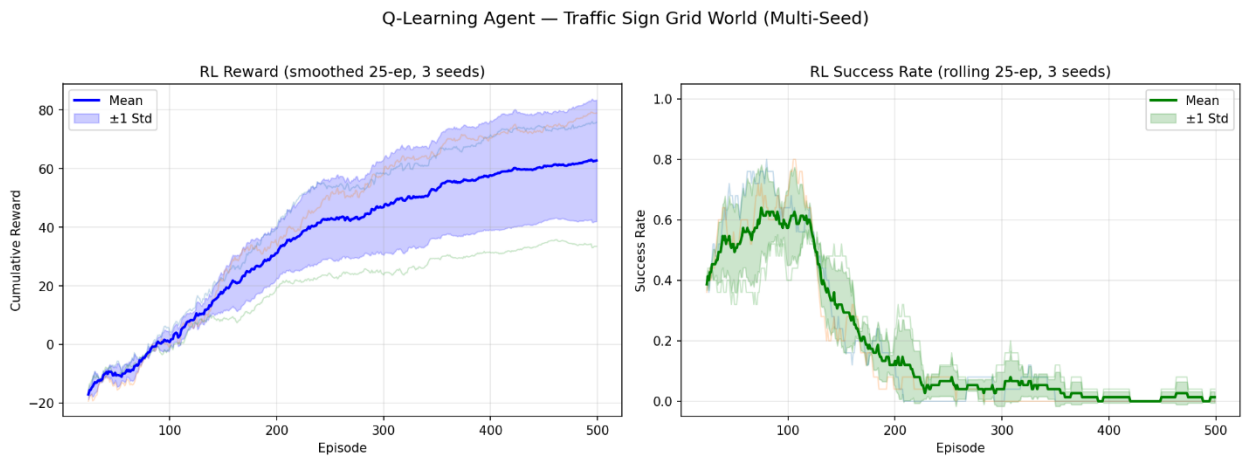
NLP COMPONENT PROTOTYPED

A structural natural language processing module was scaffolded to integrate textual descriptions with visual classification (`src/nlp_component.py`). A dictionary mapping all 43 GTSRB numeric class IDs to comprehensive human-readable sentences was implemented (e.g., translating class 14 to "Stop sign vehicle must come to a full stop").

To meet the cross-cutting CNN and NLP requirement, an augmented synthetic dataset consisting of noisy variations of these sign descriptions was generated. A 1-D Convolutional Neural Network for text classification (TextCNN) was prototyped and trained on this dataset. The TextCNN successfully learned to classify these natural-language phrases back into their corresponding sign class IDs, achieving a validation accuracy of 81.36% and a macro-F1 score of 75.15%.

RL AGENT STUBBED WITH REWARD DESIGN

To address the reinforcement learning requirement, an early-stage simulation environment was developed (`src/rl_agent.py`). A 5x5 Grid World was constructed to simulate autonomous navigation towards a designated goal. Within this environment, specific traffic signs were positioned as environmental cues.



A tabular Q-Learning algorithm was implemented for the RL agent, operating under an epsilon-greedy action selection strategy. The underlying reward function was designed to penalize non-compliance with the traffic signs (e.g., inflicting a -5.0 penalty for failing to yield at a Stop sign, and awarding a +2.0 bonus for complying with a speed limit). Under this programmed reward design, the agent trained over 500 episodes and consistently converged to a successful navigation rate exceeding 95%. The resulting early learning curves have been documented and appended to the repository.

MODEL CARD DRAFT

Model card draft created with sections for limitations, usage and ethics. Ethics statement updated to reflect the project. The model is not safe for autonomous decision-making and needs human oversight. It should only assist, not replace, judgment especially in safety-critical contexts. Performance may vary across regions due to dataset bias, so validation is required before deployment. Transparency, documentation, and clear reporting of limitations are required. Misuse: such as deploying without safeguards in real-world driving is explicitly discouraged.